此间有序（RxPal）— 技术文档

本文档旨在详细记录项目的架构设计、核心实现和技术细节，供个人学习和复盘使用。

目录

1. 项目概述
2. 技术栈详解
3. 项目目录结构
4. 架构总览
5. 数据模型设计
6. 状态管理
7. 国际化（i18n）
8. OCR 识别管线
9. 页面组件详解
10. 云函数详解
11. 样式系统
12. 构建与部署
13. 核心学习要点

---

1. 项目概述

此间有序（RxPal）

是一款面向老年用户的微信小程序，帮助管理日常用药、就诊记录和健康数据。

核心功能

功能模块	说明
首页仪表盘	展示今日服药进度、待办提醒、最近就诊记录
拍照识别	拍照或从相册选图 → OCR 识别 → 结构化存储（支持病历和药盒两种模式）
健康时间线	就诊记录时间轴 + 多指标趋势图表（Canvas 2D 折线图）

用药管理	药物列表、时间表视图、手动/拍照添加药物
个人中心	睡眠提醒、字体调节、语言切换、家人绑定、功能教程
中英双语	运行时切换，485+ 翻译键覆盖全部 UI

设计理念

- **大字体、大按钮**：所有可交互元素至少 44px 触控区域
- **温暖配色**：暖米色（#f8f5f0）底色 + 鼠尾草绿（#5b9a7d）主色调
- **引导式体验**：三步 Onboarding + 五步功能教程
- **容错设计**：OCR 失败时加载示例数据，云端不可用时本地持久化

---

2. 技术栈详解

框架层

技术	版本	作用	为什么选它
Taro	4.1.11	跨端框架	用 React 语法写微信小程序，支持编译到多端
React	18.3.0	UI 框架	组件化开发，Hooks 管理状态和副作用
TypeScript	5.4.0	类型系统	编译期类型检查，减少运行时错误

状态管理

技术	版本	作用
Zustand	4.5.0	轻量状态管理（比 Redux 简洁 10 倍）
zustand/middleware	—	`persist` 中间件实现数据持久化

后端

技术	作用
微信云开发	Serverless 后端，包含云函数、云数据库、云存储
腾讯云 OCR	GeneralBasicOCR API，通用文字识别（免费 1000 次/月）

构建工具

技术	作用
Webpack 5	模块打包（Taro 内置集成）

Sass	CSS 预处理器，支持变量、嵌套、mixin
Babel	JS/TS 编译器，支持 React JSX + 装饰器

---

3. 项目目录结构

```
cc/
├── src/                          # 前端源码
│   ├── app.ts                    # 应用入口（云开发初始化 + Onboarding 检测）
│   ├── app.config.ts             # Taro 应用配置（页面路由 + TabBar）
│   ├── app.scss                  # 全局样式（CSS 变量 + 字体缩放）
│   │
│   ├── pages/                    # 页面组件（每个页面一个文件夹）
│   │   ├── home/
│   │   │   ├── index.tsx         # 首页：仪表盘
│   │   │   └── index.scss        # 首页样式（330 行）
│   │   ├── onboarding/
│   │   │   ├── index.tsx         # 新用户引导（3 步向导）
│   │   │   └── index.scss        # 引导样式（295 行）
│   │   ├── scan/
│   │   │   ├── index.tsx         # 拍照识别（病历/处方）
│   │   │   └── index.scss        # 识别页样式（396 行）
│   │   ├── timeline/
│   │   │   ├── index.tsx         # 健康时间线（记录 + 趋势图表）
│   │   │   └── index.scss        # 时间线样式（453 行）
│   │   ├── medication/
│   │   │   ├── index.tsx         # 用药管理（列表 + 时间表）
│   │   │   ├── index.scss        # 用药页样式（206 行）
│   │   │   ├── add.tsx           # 手动添加药物
│   │   │   ├── add.scss          # 添加页样式（78 行）
│   │   │   ├── box-scan.tsx      # 药盒拍照识别
│   │   │   └── box-scan.scss     # 药盒识别样式（150 行）
│   │   └── profile/
│   │       ├── index.tsx         # 个人中心
│   │       └── index.scss        # 个人中心样式（358 行）
│   │
│   ├── components/               # 可复用组件
│   │   └── Tutorial/
```

```
│   │       ├── index.tsx          # 功能教程覆盖层（5 步引导）
│   │       └── index.scss         # 教程样式（146 行）
│   │
│   ├── utils/                     # 工具函数
│   │   ├── store.ts               # Zustand 状态管理（核心）
│   │   ├── storage.ts             # Taro Storage 适配器
│   │   ├── i18n.ts                # 国际化（中/英翻译字典 + t() 函数）
│   │   ├── cloud.ts               # 微信云开发 API 封装
│   │   └── helpers.ts             # 日期/问候语工具函数
│   │
│   ├── types/
│   │   └── index.ts               # TypeScript 接口定义
│   │
│   ├── data/
│   │   └── mock.ts                # 开发用 mock 数据
│   │
│   └── assets/                    # 静态资源（TabBar 图标等）
│
├── cloud/                         # 微信云函数
│   ├── ocr/
│   │   ├── index.js               # OCR 云函数（腾讯云 API 调用 + 文本解析）
│   │   └── package.json           # 云函数依赖
│   ├── parseRx/
│   │   └── index.js               # 处方智能解析（药品数据库匹配）
│   └── notify/
│       └── config.json            # 通知服务配置
│
├── config/                        # Taro 构建配置
│   ├── index.ts                   # 基础配置
│   ├── dev.ts                     # 开发环境
│   └── prod.ts                    # 生产环境
│
├── dist/                          # 构建输出（微信开发者工具读取此目录）
├── package.json                   # 项目依赖
├── tsconfig.json                  # TypeScript 配置
├── project.config.json            # 微信小程序项目配置
└── babel.config.js                # Babel 编译配置
```

---

4. 架构总览

应用架构图

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              微信小程序 Runtime                            │
│  ┌──────────────────────────────────────────────────────────────────────┐
│  │                              Taro 4 (React)                          │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐            │
│  │  │  Pages   │  │Components│  │  Utils   │  │  Assets  │            │
│  │  │(8 pages) │  │(Tutorial)│  │(5 mods)  │  │(icons)   │            │
│  │  └──────────┘  └──────────┘  └──────────┘  └──────────┘            │
│  │       │              │              │                                │
│  │       ▼              ▼              ▼                                │
│  │  ┌──────────────────────────────────────────────────────────────┐   │
│  │  │                Zustand Store (with persist)                  │   │
│  │  │  ┌──────────┐  ┌──────────┐  ┌──────┐  ┌──────────────┐      │   │
│  │  │  │profile   │  │meds      │  │lang  │  │reminders     │      │   │
│  │  │  │records   │  │trends    │  │      │  │(derived)     │      │   │
│  │  │  └──────────┘  └──────────┘  └──────┘  └──────────────┘      │   │
│  │  └──────────────────────────────────────────────────────────────┘   │
│  │                          │ persist                                  │
│  │                          ▼                                          │
│  │  ┌──────────────────────────────────────────────────────────────┐   │
│  │  │              Taro Storage (wx.setStorageSync)                 │   │
│  │  └──────────────────────────────────────────────────────────────┘   │
│  └──────────────────────────────────────────────────────────────────────┘
│                              │ Taro.cloud
│                              ▼
│  ┌──────────────────────────────────────────────────────────────────────┐
│  │                        WeChat Cloud Development                      │
│  │  ┌──────────────┐  ┌──────────────┐  ┌──────────────────┐          │
│  │  │Cloud Func    │  │Cloud DB      │  │Cloud Storage     │          │
│  │  │• ocr         │  │• users       │  │• OCR images      │          │
│  │  │• parseRx     │  │• records     │  │                  │          │
│  │  │• notify      │  │• meds        │  │                  │          │
│  │  └──────────────┘  └──────────────┘  └──────────────────┘          │
│  │                                                                      │
│  │                                                                      │
│  └──────────────────────────────────────────────────────────────────────┘
└──────────────────────────────────────────────────────────────────────────┘
                          │ HTTPS
                          ▼
                  ┌──────────────────────┐
                  │ Tencent Cloud        │
                  │ OCR API              │
                  │ (GeneralBasic)       │
                  └──────────────────────┘
```

数据流

```
用户操作 → React Component → useStore (Zustand action)
                                        │
                                        ├── 更新内存 state → 触发 React re-render
                                        │
                                        └── persist middleware →
Taro.setStorageSync

(自动序列化到本地)
```

页面生命周期

```
App 启动
  │
  ├── Taro.cloud.init()           # 初始化云开发
  │
  ├── 检查 onboarding_complete    # 是否完成引导
  │   ├── 否 → redirectTo → Onboarding 页面
  │   └── 是 → 进入首页
  │
  └── Zustand persist rehydrate   # 从 Storage 恢复数据
      │
      ├── 恢复 profile, medications, records, healthTrends, lang
      │
      ├── 调用 buildReminders()   # 根据当前药物生成今日提醒
      │
      └── 调用 updateTabBarLabels()  # 根据 lang 更新 TabBar
```

---

5. 数据模型设计

核心 TypeScript 接口

文件位置：`src/types/index.ts`

UserProfile — 用户档案

```
interface UserProfile {
  name: string              // 用户姓名（onboarding 时设置）
  age: number               // 年龄
  avatar: string            // 头像 emoji（'🧓' | '👴' | '🧑' | '👩'）
  sleepReminderTime: string // 睡眠提醒时间（如 '21:30'）
  fontSize: 'normal' | 'large' | 'xlarge'  // 字体档位
}
```

设计要点： - avatar 使用 emoji 而非图片 URL，省去图片上传/存储，适合老年用户审美 - fontSize 只有三档而非自由缩放，减少选择困难

Medication — 药物

```
interface Medication {
  id: string            // 唯一标识（'med-xxx'）
  name: string          // 药名（如 '氨氯地平片'）
  dosage: string        // 剂量（如 '5mg'）
  frequency: string     // 频率描述（如 '每日1次'）
  times: string[]       // 具体服药时间点（如 ['08:00', '18:00']）
  duration: string      // 疗程（如 '7天'、'30天'、'长期'）
  remainingDays: number // 剩余天数
  notes: string         // 用药备注（如 '饭后服用'）
  active: boolean       // 是否启用
}
```

设计要点： - times 是字符串数组而非单个 cron 表达式，因为老年用户的用药时间点明确（早中晚睡前），无需复杂调度 - remainingDays 用于前端显示剩余量告警（<7 天显示红色） - frequency 是人类可读字符串，不是枚举，方便 OCR 结果直接填入

Reminder — 提醒

```
interface Reminder {
  id: string                        // 唯一标识（格式：'rem-{medId}-{index}' 或 'rem-sleep'）
  type: 'medication' | 'sleep'      // 提醒类型
  time: string                      // 提醒时间（如 '08:00'）
  title: string                     // 标题（药名 或 '该休息啦'）
  subtitle?: string                 // 副标题（剂量·备注）
  medicationId?: string             // 关联药物 ID
  done: boolean                     // 是否已完成
  skipped: boolean                  // 是否已跳过
}
```

设计要点： - done 和 skipped 是互斥的（跳过时 done=false，完成时 skipped=false） - Reminder 不持久化到 Storage，每次启动根据当前 medications

重新生成，避免过期提醒残留 - `id` 包含 medId 和时间点 index，确保同一药物不同时间点的提醒可独立操作

MedicalRecord — 就诊记录

```
interface MedicalRecord {
  id: string
  date: string              // 就诊日期（'YYYY-MM-DD'）
  hospital: string          // 医院名称
  department: string        // 科室
  doctor: string            // 主治医生
  diagnosis: string         // 诊断
  details: string           // 详细描述
  vitals?: {                // 生命体征（可选）
    bloodPressureSys?: number
    bloodPressureDia?: number
    bloodSugar?: number
    heartRate?: number
    weight?: number
  }
  prescriptions: Prescription[]  // 关联处方
}
```

HealthTrend — 健康趋势数据点

```
interface HealthTrend {
  date: string                    // 日期（'YYYY-MM-DD'）
  bloodPressureSys?: number       // 收缩压
  bloodPressureDia?: number       // 舒张压
  bloodSugar?: number             // 血糖（mmol/L）
  heartRate?: number              // 心率（bpm）
  weight?: number                 // 体重（kg）
}
```

设计要点： - 所有指标都是 optional，一次记录不必填全 - 相同日期的数据会 merge（`addHealthData` action 中实现）

FamilyMember — 家人

```
interface FamilyMember {
  id: string
  name: string
  relation: string      // 关系（如 '孙子'、'女儿'）
  avatar: string         // emoji 头像
  phone: string          // 手机号
```

```
  bound: boolean          // 是否已绑定
  lastSeen?: string       // 最近在线时间
}
```

---

6. 状态管理：Zustand + Persist

文件位置：`src/utils/store.ts`、`src/utils/storage.ts`

6.1 为什么选 Zustand

对比项	Redux	Zustand	选择理由
样板代码	大量（action/reducer/selector）	极少（一个 create 调用）	小项目无需复杂架构
中间件	需要 redux-persist 等外部库	内置 persist 中间件	开箱即用
学习成本	高（概念多：store/dispatch/reducer）	低（就是一个 hook）	快速开发
包体积	~7KB	~1.5KB	小程序包体积敏感

6.2 Storage 适配器

```
// src/utils/storage.ts
import Taro from '@tarojs/taro'
import type { StateStorage } from 'zustand/middleware'

export const taroStorage: StateStorage = {
  getItem: (name) => {
    const value = Taro.getStorageSync(name)
    return value || null
  },
  setItem: (name, value) => {
    Taro.setStorageSync(name, value)
  },
  removeItem: (name) => {
    Taro.removeStorageSync(name)
  },
}
```

为什么需要适配器： - Zustand 的 `persist` 中间件默认使用 `window.localStorage` - 微信小程序没有 `window` 对象，需要用 `Taro.getStorageSync` / `Taro.setStorageSync` - `StateStorage` 是 Zustand 定义的接口，只要实现 `getItem`、`setItem`、`removeItem` 三个方法即可

技术细节： - 使用 `Sync` 版本（同步 API）而非异步版本，因为 Zustand persist 的 `createJSONStorage` 对同步 storage 支持更好 - `getItem` 返回 `null` 而非 `undefined`，符合 `StateStorage` 接口要求

6.3 Store 完整结构

```
export const useStore = create<AppState>()(
  persist(
    (set, get) => ({
      // ====== State ======
      profile: mockProfile,           // 初始值来自 mock（首次启动时使用）
      records: mockRecords,
      medications: mockMeds,
      reminders: buildReminders(...), // 派生状态，不持久化
      family: mockFamily,
      healthTrends: [...mockTrends],
      lang: 'zh' as Lang,

      // ====== Actions ======
      toggleReminder: (id) => set((s) => ({ ... })),
      skipReminder: (id) => set((s) => ({ ... })),
      updateSleepTime: (time) => set((s) => ({ ... })),
      updateAvatar: (avatar) => set((s) => ({ ... })),
      updateProfile: (updates) => set((s) => ({ ... })),
      updateFontSize: (size) => set((s) => ({ ... })),
      addRecord: (record) => set((s) => ({ ... })),
      addHealthData: (data) => set((s) => ({ ... })),
      setLang: (lang) => { set({ lang }); updateTabBarLabels(lang) },
      addMedication: (med) => set((s) => ({ ... })),
      updateMedication: (id, updates) => set((s) => ({ ... })),
      removeMedication: (id) => set((s) => ({ ... })),
      regenerateReminders: () => set((s) => ({ ... })),
    }),

    // ====== Persist 配置 ======
    {
      name: 'rxpal-store',                            // Storage key 名称
      storage: createJSONStorage(() => taroStorage),   // 使用 Taro 适配器
      partialize: (state) => ({                        // 只持久化这些字段
        profile: state.profile,
        medications: state.medications,
        records: state.records,
        healthTrends: state.healthTrends,
        lang: state.lang,
      }),
      onRehydrateStorage: () => (state) => {           // 数据恢复后的回调
        if (state) {
```

```
          state.reminders = buildReminders(state.medications,
state.profile.sleepReminderTime)
          updateTabBarLabels(state.lang)
        }
      },
    }
  )
)
```

6.4 Persist 生命周期详解

```
App 启动
  │
  ├── 1. Zustand 创建 store，使用 mock 数据初始化 state
  │        （此时 state = mockProfile + mockMeds + ...）
  │
  ├── 2. persist 中间件读取 Taro.getStorageSync('rxpal-store')
  │       │
  │       ├── 无数据 → 保持 mock 初始值（首次安装）
  │       │
  │       └── 有数据 → 解析 JSON，merge 到 state
  │                    （此时 state = 用户真实数据）
  │
  ├── 3. onRehydrateStorage 回调执行
  │       ├── buildReminders() — 从真实药物数据生成今日提醒
  │       └── updateTabBarLabels() — 根据语言设置 TabBar
  │
  └── 4. React 组件 re-render，显示真实用户数据
```

关键点： - partialize 排除了 reminders 和 family，因为 reminders 每天需要重新生成，family 来自云端 - onRehydrateStorage 返回的是一个回调函数（注意双层函数），这是 Zustand 的 API 设计

6.5 buildReminders() — 提醒生成器

```
function buildReminders(medications: Medication[], sleepTime: string):
Reminder[] {
  const reminders: Reminder[] = []
  const now = new Date()
  const currentTime = `${now.getHours().toString().padStart(2,
'0')}:${now.getMinutes().toString().padStart(2, '0')}`

  // 为每个活跃药物的每个时间点生成一条提醒
  medications.filter(m => m.active).forEach(med => {
    med.times.forEach((time, idx) => {
```

```
      reminders.push({
        id: `rem-${med.id}-${idx}`,        // 唯一 ID = 药物ID + 时间点索引
        type: 'medication',
        time,
        title: med.name,
        subtitle: `${med.dosage} · ${med.notes}`,
        medicationId: med.id,
        done: time < currentTime && Math.random() > 0.3,  //
过去时间随机标记已完成
        skipped: false,
      })
    })
  })

  // 添加睡眠提醒
  reminders.push({
    id: 'rem-sleep',
    type: 'sleep',
    time: sleepTime,
    title: '该休息啦',
    subtitle: '早睡早起身体好',
    done: false,
    skipped: false,
  })

  return reminders.sort((a, b) => a.time.localeCompare(b.time))
}
```

6.6 addMedication() — 药物增删联动提醒

```
addMedication: (med) =>
  set((s) => {
    const medications = [...s.medications, med]
    return {
      medications,
      reminders: buildReminders(medications, s.profile.sleepReminderTime)
    }
  }),
```

设计模式：

每次增删药物都自动重新生成全部提醒，而非手动维护提醒列表。虽然全量重建看起来“浪费”，但药物数量通常只有 5-10 种，性能完全不是问题，代码却简洁得多。

6.7 updateTabBarLabels() — 运行时更新 TabBar

```
function updateTabBarLabels(lang: Lang) {
  const labels = {
    zh: ['首页', '时间线', '识别', '用药', '我的'],
```

```
    en: ['Home', 'Timeline', 'Scan', 'Meds', 'Me'],
  }
  labels[lang].forEach((text, index) => {
    Taro.setTabBarItem({ index, text })
  })
}
```

为什么需要这个函数： - `app.config.ts` 中的 TabBar 配置是静态的，编译时确定 - 微信小程序不支持根据条件编译动态修改 TabBar 文字 - `Taro.setTabBarItem()` 是微信提供的运行时 API，可以动态修改标签文字 - 调用时机：语言切换时（`setLang` action）和应用启动恢复数据后（`onRehydrateStorage`）

---

7. 国际化（i18n）

文件位置：`src/utils/i18n.ts`

7.1 架构设计

```
┌──────────────────────────────────┐
│          i18n.ts                 │
│                                  │
│  zh: { 'key': '中文值', ... }     │  ← 翻译字典（485+ 键）
│  en: { 'key': 'English', ... }   │
│                                  │
│  t(key, params?) → string        │  ← 翻译函数（非 Hook）
│  tArray(key) → string[]          │  ← 数组翻译（状态消息等）
│  useT() → { t, tArray, lang }    │  ← React Hook（触发重渲染）
│                                  │
└──────────────────────────────────┘
           │ reads lang from
           ▼
┌──────────────────────┐
│   Zustand Store      │
│   state.lang = 'zh'  │
└──────────────────────┘
```

7.2 翻译字典结构

```
type TranslationDict = Record<string, string | string[]>

const zh: TranslationDict = {
  // 简单字符串
  'app.name': '此间有序',
  'greeting.morning': '早上好',
```

```
  // 带参数模板
  'home.pendingItems': '{n}项待办',
  'date.format': '{year}年{month}月{day}日 星期{weekday}',

  // 字符串数组（用于随机/序列显示）
  'date.weekdays': ['日', '一', '二', '三', '四', '五', '六'],
  'scan.status': ['正在上传图片…', '准备就绪…', ...],  // 17 条状态消息
  'scan.quotes': ['健康是最好的财富…', ...],            // 10 条暖心语录
}
```

命名约定： {页面/模块}.{语义} — 如 home.todayProgress、scan.camera、ob.welcome

7.3 t() 函数 — 翻译核心

```
export function t(key: string, params?: Record<string, string | number>):
string {
  const lang = useStore.getState().lang || 'zh'          // ① 非 Hook
方式读状态
  const val = dicts[lang][key] ?? dicts['zh'][key] ?? key  // ②
查字典，fallback 到中文，再 fallback 到 key

  if (Array.isArray(val)) return val[0] || key           // ③ 数组取第一个元素
  if (!params) return val                                // ④ 无参数直接返回
  return val.replace(/\{(\w+)\}/g, (_, k) =>             // ⑤ 替换模板占位符
    String(params[k] ?? `{${k}}`)
  )
}
```

关键设计决策：

1. **为什么 t() 不是 Hook？**
   - t() 使用 useStore.getState().lang（直接读取），而非 useStore(s => s.lang)（订阅）
   - 这意味着 t() 可以在任何地方调用（事件处理器、工具函数），不受 Hook 规则限制
   - 但这也意味着语言切换时，调用 t() 的组件不会自动重渲染
2. **useT() 解决重渲染问题：**

```
export function useT() {
  const lang = useStore(s => s.lang)  // ← Hook 方式订阅 lang
```

```
  return { t, tArray, lang }
}
```

组件中使用 `const { t } = useT()`，当 `lang` 变化时组件重渲染，`t()` 被重新调用，返回新语言的文本。

3. **三级 fallback**：当前语言字典 → 中文字典 → `key` 本身
   - 确保即使翻译缺失也不会白屏
   - 开发时只要看到原始 key 就知道漏翻译了

7.4 `tArray()` 函数 — 获取字符串数组

```
export function tArray(key: string): string[] {
  const lang = useStore.getState().lang || 'zh'
  const val = dicts[lang][key] ?? dicts['zh'][key]
  if (Array.isArray(val)) return val
  return []
}
```

使用场景：- `tArray('scan.status')` — 获取 17 条 OCR 进度状态消息 - `tArray('scan.quotes')` — 获取 10 条暖心语录 - `tArray('date.weekdays')` — 获取星期名称数组

7.5 组件中的使用方式

```
// 页面组件中
function HomePage() {
  const { t } = useT()  // ← 订阅 lang 变化

  return (
    <View>
      <Text>{t('home.todayProgress')}</Text>           {/* 今日服药进度 */}
      <Text>{t('home.pendingItems', { n: 3 })}</Text>  {/* 3项待办 */}
      <Text>{t('home.allDone')}</Text>                 {/* 今天的任务都完成啦！
*/}
    </View>
  )
}
```

7.6 语言切换流程

```
用户点击 "English" 按钮
  │
  ├── 调用 store.setLang('en')
```

```
  │      │
  │      ├── set({ lang: 'en' })              → Zustand 更新 state
  │      │     │
  │      │     └── persist middleware          → 自动写入 Storage
  │      │
  │      └── updateTabBarLabels('en')          → 调用 5 次
Taro.setTabBarItem()
  │
  └── 所有使用 useT() 的组件重新渲染
        │
        └── t('key') 读取 en 字典 → 显示英文
```

---

8. OCR 识别管线

8.1 完整流程图

```
前端                                  云端                          第三方
────                                  ────                          ──────

1. 用户拍照/选择图片
      │
2. Taro.chooseImage()
      │ 获取 filePath
      ▼
3. callOcr(filePath, mode?)
      │
      ├── Taro.cloud.uploadFile()  ──→  云存储（ocr/timestamp.jpg）
      │                                      │
      │                                      ▼
      ├── Taro.cloud.callFunction() ──→  cloud/ocr/index.js
      │                                      │
      │                                      ├── cloud.downloadFile() ←
从云存储下载
      │                                      │
      │                                      ├── Base64 编码
      │                                      │
      │                                      ├── OcrClient.GeneralBasicOCR()
──→ 腾讯云 OCR API
      │                                      │
│
      │                                      │   ←── ocrResult.TextDetections
←──────┘
      │                                      │
```

```
      │                                        ├── 拼接全部识别文本
      │                                        │
      │                                        ├── mode === 'box'
      │                                        │    ├── 是 → parseBoxText(text)
      │                                        │    └── 否 → parseOcrText(text)
      │                                        │
      │    ◀── { success, text, parsed } ◀────┘
      │
4. 前端收到结果
      │
      ├── 成功 → 显示可编辑结果表单
      │          用户确认后保存到 store
      │
      └── 失败 → 显示 mock 数据
                 提示"云端识别暂不可用"
```

8.2 云函数代码详解

文件位置：`cloud/ocr/index.js`

初始化

```
const cloud = require('wx-server-sdk')
const tencentcloud = require('tencentcloud-sdk-nodejs')
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })

const OcrClient = tencentcloud.ocr.v20181119.Client
```

- `wx-server-sdk`：微信云函数 SDK，提供 `cloud.downloadFile()` 等能力
- `tencentcloud-sdk-nodejs`：腾讯云 Node.js SDK，提供 OCR API 客户端
- `DYNAMIC_CURRENT_ENV`：自动使用当前云开发环境，无需硬编码环境 ID

主函数

```
exports.main = async (event) => {
  const { fileID, mode } = event

  // 1. 从云存储下载图片
  const fileRes = await cloud.downloadFile({ fileID })
  const imgBase64 = fileRes.fileContent.toString('base64')

  // 2. 创建 OCR 客户端（密钥从环境变量读取）
  const client = new OcrClient({
    credential: {
      secretId: process.env.TENCENT_SECRET_ID,
      secretKey: process.env.TENCENT_SECRET_KEY,
```

```
    },
    region: 'ap-guangzhou',
    profile: { httpProfile: { endpoint: 'ocr.tencentcloudapi.com' } },
  })

  // 3. 调用通用文字识别
  const ocrResult = await client.GeneralBasicOCR({
    ImageBase64: imgBase64,
    LanguageType: 'zh',
  })

  // 4. 拼接文本 + 结构化解析
  const text = ocrResult.TextDetections.map(item =>
item.DetectedText).join('\n')
  const parsed = mode === 'box' ? parseBoxText(text) : parseOcrText(text)

  return { success: true, text, parsed }
}
```

安全设计： - API 密钥存储在云函数环境变量中，不在代码里硬编码 - 密钥不会暴露到前端（前端只调用云函数，不直接调用腾讯云 API）

parseOcrText(text) — 病历/处方解析

解析策略：逐行扫描 + 正则匹配

```
for (const line of lines) {
  // 文档类型判断
  if (line.includes('处方') || line.includes('Rx')) result.type =
'prescription'

  // 各字段提取（示例：日期提取）
  if (line.match(/\d{4}[-/.]\d{1,2}[-/.]\d{1,2}/) && !result.date) {
    result.date = line.match(/\d{4}[-/.]\d{1,2}[-
/.]\d{1,2}/)[0].replace(/[/.]/g, '-')
  }

  // 药品行匹配（包含剂量单位关键词）
  if (line.match(/(mg|ml|片|粒|颗|g|μg|ug|丸|袋|支)\b/i)) {
    const medMatch =
line.match(/^[.\d\s]*(.+?)\s+(\d+\s*(?:mg|ml|g|μg|ug))/i)
    if (medMatch) {
      result.medications.push({ name: medMatch[1], dosage: medMatch[2], ...
})
    }
```

```
  }
}
```

解析字段：

字段	匹配规则	示例输入
类型	包含 '处方' 或 'Rx'	"处方笺" → type = 'prescription'
姓名	姓名[：:] + 文本	"姓名：王建国" → "王建国"
日期	YYYY-MM-DD 或 YYYY/MM/DD	"2025-03-01"
医院	包含 '医院' 的行	"北京协和医院"
科室	包含 '科室' 或 '内科/外科' 等	"心内科"
医生	包含 '医生' 或 '医师'	"主治医师：李明" → "李明"
诊断	包含 '诊断' 的行	"诊断：高血压2级"
药品	包含剂量单位的行	"氨氯地平 5mg"

parseBoxText(text) — 药盒解析

解析策略：按【标签】分段提取

药盒通常格式：

```
┌──────────────────────────┐
│  阿莫西林胶囊              │  ← 药品名称
│  【通用名称】阿莫西林胶囊   │
│  【规格】0.5g              │
│  【用法用量】口服。成人一次  │
│  0.5g，一日3次。           │
└──────────────────────────┘
```

```
function parseBoxText(text) {
  // 1. 提取药品名称
  const nameMatch =
text.match(/(?:通用名称|药品名称|商品名)[】：:\s]*([^\n【]+)/)
  // fallback：取第一行非空文本

  // 2. 提取用法用量
  const usageMatch = text.match(/用法用量[】：:\s]*([\s\S]*?)(?=【|$)/)
  // 从用法用量段中提取：
  //   剂量："一次5mg"   → regex: /[一每]次\s*(\d+\.?\d*\s*(?:mg|ml|g|片|...))/
  //   频率："一日3次"   → regex: /[一每](?:日|天)\s*(\d+)\s*次/
```

```
  //    备注：饭后/饭前/空腹/睡前

  // 3. 根据频率设置默认时间
  if (n === '1') result.times = ['08:00']
  else if (n === '2') result.times = ['08:00', '18:00']
  else if (n === '3') result.times = ['08:00', '12:00', '18:00']

  // 4. fallback：从【规格】提取剂量
  if (!result.dosage) {
    const specMatch = text.match(/规格[】：:\s]*([^\n【]+)/)
  }
}
```

8.3 前端交互：三阶段模式

```
Stage 1: capture          Stage 2: loading          Stage 3: result
┌──────────────┐          ┌──────────────┐          ┌──────────────┐
│              │          │              │          │ ✅ 识别完成   │
│   取景框     │  拍照    │ ▓▓▓▓▓▓▓▓ 60% │  完成    │              │
│              │ ──→      │              │ ──→      │ [可编辑表单] │
│              │          │ "正在识别…"  │          │              │
│ [📷] [🖼]    │          │ "健康名言…"  │          │ [确认保存]   │
└──────────────┘          └──────────────┘          └──────────────┘
```

加载阶段 UX 设计： - 进度条从 0% 到 95%（不到 100%，避免用户以为卡住了） - 状态文字轮播（17 条消息，根据进度范围选取） - 暖心语录随机显示（10 条养生格言） - 即使 OCR 只需 2 秒，也保证最少 3 秒加载时间（让用户感觉“认真在处理”）

8.4 前端调用代码

```
// src/utils/cloud.ts
export async function callOcr(filePath: string, mode?: 'box') {
  if (!Taro.cloud) return null
  try {
    // 1. 上传图片到云存储
    const uploadRes = await Taro.cloud.uploadFile({
      cloudPath: `ocr/${Date.now()}.jpg`,  // 用时间戳避免重名
      filePath,
    })

    // 2. 调用 OCR 云函数
    const { result } = await Taro.cloud.callFunction({
      name: 'ocr',
      data: { fileID: uploadRes.fileID, mode },
    })
```

```
    return result
  } catch (e) {
    console.error('OCR error:', e)
    return null
  }
}
```

---

9. 页面组件详解

9.1 应用入口 — app.ts

```
function App({ children }: PropsWithChildren) {
  useLaunch(() => {
    // 1. 初始化微信云开发
    if (Taro.cloud) {
      Taro.cloud.init({ traceUser: true })
    }
    // 2. 新用户引导检测
    const onboardingComplete = Taro.getStorageSync('onboarding_complete')
    if (!onboardingComplete) {
      Taro.redirectTo({ url: '/pages/onboarding/index' })
    }
  })
  return children
}
```

`useLaunch` vs `useEffect`： - `useLaunch` 是 Taro 提供的 Hook，对应小程序的 `App.onLaunch` 生命周期 - 只在应用冷启动时执行一次，热启动（从后台恢复）不会触发 - 比 `useEffect` 更早执行，适合做初始化和路由守卫

9.2 首页 — pages/home/index.tsx

核心 UI 结构：

```
┌──────────────────────────┐
│  👴 早上好，王建国        │  ← 问候卡片
│  2025年3月1日 星期六      │
├──────────────────────────┤
│  ▓▓▓
```

```
│  今日提醒 (3项待办)          │
│  ✅ 08:00 氨氯地平 5mg      │  ← 提醒列表
│  ☐ 12:00 阿莫西林 500mg     │
│  ☐ 21:30 该休息啦           │
├──────────────────────────┤
│  最近就诊                   │
│  3/1 北京协和医院 心内科     │
└──────────────────────────┘

                [📷]          ← 悬浮拍照按钮
```

关键逻辑：

```
// 计算服药进度
const totalMeds = reminders.filter(r => r.type === 'medication').length
const doneMeds = reminders.filter(r => r.type === 'medication' &&
r.done).length
const progress = totalMeds > 0 ? Math.round((doneMeds / totalMeds) * 100) : 0

// 获取待办提醒（未完成 + 未跳过）
const upcoming = reminders.filter(r => !r.done && !r.skipped)
```

9.3 引导页 — pages/onboarding/index.tsx

三步向导：

```
Step 0: 欢迎              Step 1: 设置信息          Step 2: 完成

┌──────────┐            ┌──────────┐            ┌──────────┐
│ 此间有序  │            │ 您的称呼  │            │ ✨ 设置完成！│
│          │            │ [_______] │            │          │
│ 📷 拍照识别 │            │ 您的年龄  │            │ 👴 王建国  │
│ 💊 用药提醒 │            │ [_______] │            │ 72 岁    │
│ 📊 趋势追踪 │            │          │            │          │
│ 👨‍👩‍👧 远程关怀 │            │ 选择头像  │            │ 接下来有教程 │
│          │            │ [👴][👵]  │            │ 帮您了解功能 │
│ [中文][Eng] │            │ [🧑][👩]  │            │          │
│          │            │          │            │ [返回][进入] │
│ [开始使用] │            │ [上一步][下一步]│            └──────────┘
└──────────┘            └──────────┘
```

语言选择实现：

```
<View className='ob-lang'>
  <Text className='ob-lang-label'>{t('ob.langLabel')}</Text>
  <View className='ob-lang-options'>
    <View
      className={`ob-lang-btn ${lang === 'zh' ? 'ob-lang-btn-active' : ''}`}
      onClick={() => setLang('zh')}
    >
      中文
    </View>
    <View
      className={`ob-lang-btn ${lang === 'en' ? 'ob-lang-btn-active' : ''}`}
      onClick={() => setLang('en')}
    >
      English
    </View>
  </View>
</View>
```

完成时保存：

```
const handleFinish = () => {
  updateProfile({ name, age: Number(age) })
  Taro.setStorageSync('onboarding_complete', true)
  Taro.setStorageSync('show_tutorial', true)
  Taro.switchTab({ url: '/pages/home/index' })
}
```

9.4 拍照识别 — pages/scan/index.tsx

状态机：

```
type Stage = 'capture' | 'loading' | 'result'
const [stage, setStage] = useState<Stage>('capture')
```

加载阶段进度控制：

```
// 进度条模拟（独立于实际 OCR 进度）
useEffect(() => {
  if (stage !== 'loading') return
  const timer = setInterval(() => {
    setProgress(prev => {
      if (prev >= 95) { clearInterval(timer); return 95 }
      return prev + Math.random() * 8 + 2  // 每次增加 2-10%
    })
  }, 400)
  return () => clearInterval(timer)
}, [stage])
```

状态消息选取（根据进度范围）：

```
function getStatusMsg(progress: number): string {
  const msgs = tArray('scan.status')  // 17 条消息
  const idx = Math.min(Math.floor(progress / 6), msgs.length - 1)
  return msgs[idx]
}
// progress 0-5% → msgs[0] "正在上传图片…"
// progress 6-11% → msgs[1] "准备就绪…"
// progress 90-95% → msgs[16] "马上就好…"
```

结果编辑表单：

```
// 可切换类型：record（病历）/ prescription（处方）
const [resultType, setResultType] = useState<'record' |
'prescription'>('record')

// 可编辑字段
const [patientName, setPatientName] = useState('')
const [visitDate, setVisitDate] = useState('')
const [hospital, setHospital] = useState('')
const [department, setDepartment] = useState('')
const [doctor, setDoctor] = useState('')
const [diagnosis, setDiagnosis] = useState('')
const [resultMeds, setResultMeds] = useState<any[]>([])  //
药品列表，每项可编辑
```

9.5 时间线 — pages/timeline/index.tsx

两个 Tab：

[就诊记录] [趋势分析]

趋势图表 — Canvas 2D 实现：

```
// 4 个可选指标
const metricChips = [
  { key: 'bp', label: t('timeline.bp') },       // 血压
  { key: 'bs', label: t('timeline.bs') },       // 血糖
  { key: 'hr', label: t('timeline.hr') },       // 心率
  { key: 'wt', label: t('timeline.wt') },       // 体重
]
```

Canvas 绑定流程：

```
useEffect(() => {
  // 1. 获取 Canvas 节点
```

```
  const query = Taro.createSelectorQuery()
  query.select('#trendChart')
    .fields({ node: true, size: true })
    .exec((res) => {
      const canvas = res[0].node
      const ctx = canvas.getContext('2d')

      // 2. 设置 DPR
      const dpr = Taro.getSystemInfoSync().pixelRatio
      canvas.width = res[0].width * dpr
      canvas.height = res[0].height * dpr
      ctx.scale(dpr, dpr)

      // 3. 绘制图表
      drawChart(ctx, chartWidth, chartHeight)
    })
}, [selectedMetric, healthTrends])
```

图表绘制逻辑：

1. 计算数据范围（min/max + padding）
2. 绘制背景网格线（5 条水平线）
3. 绘制 Y 轴标签
4. 绘制 X 轴日期标签
5. 对每个数据系列：
   a. 绘制渐变填充区域（line → bottom → close path → fill）
   b. 绘制折线（stroke）
   c. 绘制数据点（fillArc）
6. 绘制图例

9.6 用药管理 — pages/medication/index.tsx

列表视图：

```
┌──────────────────────────────┐
│  氨氯地平片                    │
│  5mg · 每日1次                 │
│  ⏰ 08:00                      │
│  ▓▓▓
```

```
│  10mg · 每日1次                    │
│  ⏰ 08:00                          │
│  ██▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒ 3天  │  ← 红色（<7天）
│  ⚠️ 仅余3天                        │
└────────────────────────────────────┘
```

时间表视图：

```
  08:00          12:00          18:00          21:00
    ●──────────────●──────────────●──────────────●
  早晨           中午           傍晚           睡前

 ┌──────┐       ┌──────┐       ┌──────┐       ┌──────┐
 │氨氯地平│       │（无药）│       │二甲双胍│       │阿托伐他汀│
 │二甲双胍│       │      │       │      │       │      │
 │阿司匹林│       │      │       │      │       │      │
 │氯雷他定│       └──────┘       └──────┘       └──────┘
 └──────┘
```

剩余天数颜色逻辑：

```
const getStockColor = (days: number) => {
  if (days <= 7) return 'coral'   // 紧急 — 红色
  if (days <= 14) return 'amber'  // 警告 — 黄色
  return 'sage'                   // 正常 — 绿色
}
```

9.7 手动添加药物 — pages/medication/add.tsx

表单设计：

```
┌──────────────────────────────────┐
│  药物名称                          │
│  [_________________]              │
├──────────────────────────────────┤
│  剂量                              │
│  [______] + 预设按钮               │
│  [5mg][10mg][25mg]                │
│  [50mg][100mg][500mg]             │
├──────────────────────────────────┤
│  服药频率                          │
│  [每日1次] [每日2次]               │
│  [每日3次]                         │
├──────────────────────────────────┤
│  服药时间                          │
```

```
│  [🌅早晨] [☀️中午]        │
│  [🌆傍晚] [🌙睡前]        │
├──────────────────────────┤
│  疗程                     │
│  [7天] [14天] [30天]      │
│  [长期]                   │
├──────────────────────────┤
│  备注（可选）              │
│  [_________________]     │
├──────────────────────────┤
│      [ 保存药物 ]         │
└──────────────────────────┘
```

保存逻辑：

```
const handleSave = () => {
  if (!name.trim()) {
    Taro.showToast({ title: t('medAdd.nameRequired'), icon: 'none' })
    return
  }
  addMedication({
    id: `med-${Date.now()}`,      // 用时间戳生成唯一 ID
    name: name.trim(),
    dosage,
    frequency,
    times: selectedTimes,
    duration,
    remainingDays: duration === '长期' ? 999 : parseInt(duration),
    notes,
    active: true,
  })
  Taro.showToast({ title: t('boxScan.saved'), icon: 'success' })
  setTimeout(() => Taro.navigateBack(), 1500)
}
```

9.8 药盒识别 — pages/medication/box-scan.tsx

与 scan/index.tsx 结构相同（三阶段），差异：- 调用 callOcr(filePath, 'box') — 传入 mode: 'box' - 结果是单个药物（非完整就诊记录） - 确认后调用 addMedication() 而非 addRecord()

9.9 个人中心 — pages/profile/index.tsx

可折叠分区设计：

```
const [expandedSection, setExpandedSection] = useState<string>('reminders')

// 点击标题切换展开/收起
const toggleSection = (section: string) => {
  setExpandedSection(prev => prev === section ? '' : section)
}
```

语言切换 UI：

```
<View className='profile-setting-row'>
  <Text>{t('profile.language')}</Text>
  <View className='profile-font-options'>
    <View
      className={`profile-font-btn ${lang === 'zh' ? 'active' : ''}`}
      onClick={() => setLang('zh')}
    >中文</View>
    <View
      className={`profile-font-btn ${lang === 'en' ? 'active' : ''}`}
      onClick={() => setLang('en')}
    >English</View>
  </View>
</View>
```

9.10 功能教程 — components/Tutorial/index.tsx

覆盖层设计：

```
// 全屏半透明覆盖层
<View className='tutorial-overlay'>
  <View className='tutorial-card'>
    {/* 步骤徽章 */}
    <View className='tutorial-badge'>{step + 1}/{STEPS.length}</View>

    {/* emoji + 标题 + 描述 */}
    <Text className='tutorial-emoji'>{STEPS[step].emoji}</Text>
    <Text className='tutorial-title'>{STEPS[step].title}</Text>
    <Text className='tutorial-desc'>{STEPS[step].desc}</Text>

    {/* 进度点 */}
    <View className='tutorial-dots'>
      {STEPS.map((_, i) => (
        <View className={`tutorial-dot ${i === step ? 'active' : ''}`} />
      ))}
    </View>

    {/* 导航按钮 */}
```

```
    <View className='tutorial-actions'>
      {step > 0 && <Button onClick={prev}>{t('tutorial.prev')}</Button>}
      <Button onClick={step < 4 ? next : finish}>
        {step < 4 ? t('tutorial.next') : t('tutorial.start')}
      </Button>
    </View>
  </View>
</View>
```

---

10. 云函数详解

10.1 cloud/ocr/index.js

完整代码解析见 第 8 节

依赖：

```
{
  "dependencies": {
    "wx-server-sdk": "~2.6.3",
    "tencentcloud-sdk-nodejs": "^4.0.0"
  }
}
```

环境变量： - `TENCENT_SECRET_ID` — 腾讯云 API 密钥 ID - `TENCENT_SECRET_KEY` — 腾讯云 API 密钥 Key

API 配额： GeneralBasicOCR 每月免费 1000 次

10.2 cloud/parseRx/index.js

功能： 将 OCR 识别出的药品名与内置药品数据库匹配，补全频率、用法等信息。

药品数据库（DRUG_DB）示例：

```
const DRUG_DB = {
  '氨氯地平': { category: '降压药', frequency: '每日1次', times: ['08:00'] },
  '二甲双胍': { category: '降糖药', frequency: '每日2次', times: ['08:00',
'18:00'] },
  '阿托伐他汀': { category: '降脂药', frequency: '每日1次', times: ['21:00'] },
  // ... 更多药品
}
```

匹配逻辑： 对 OCR 提取的药名进行模糊匹配（去除“片”、“胶囊”等后缀），命中数据库则自动补全默认用法。

10.3 cloud/notify/

当前状态： 配置文件占位，功能未实现。

设计意图： 通过微信模板消息/订阅消息向用户推送服药提醒和家人关怀通知。

---

11. 样式系统

11.1 设计 Token

```
// 全局 CSS 变量（app.scss 中定义）

// 主色调
$sage: #5b9a7d;          // 鼠尾草绿 — 主操作色
$cream: #f8f5f0;         // 暖米色 — 背景色
$coral: #e8736a;         // 珊瑚红 — 警告/紧急
$amber: #f0a946;         // 琥珀黄 — 提醒/警告
$sky: #5fa6d9;           // 天蓝 — 信息/辅助

// 文字
$text-primary: #333333;
$text-secondary: #666666;
$text-muted: #999999;

// 圆角
$radius-sm: 8px;
$radius-md: 16px;
$radius-lg: 24px;

// 字体缩放（通过 CSS 变量实现三档切换）
// --font-scale: 1 | 1.2 | 1.4
```

11.2 字体缩放实现

```
// app.scss
.font-normal { --font-scale: 1; }
.font-large { --font-scale: 1.2; }
.font-xlarge { --font-scale: 1.4; }
```

```
// 各组件中使用
.card-title {
  font-size: calc(32rpx * var(--font-scale, 1));
}
```

rpx 是什么： - 微信小程序的响应式像素单位 - 规定屏幕宽度 = 750rpx - 在 iPhone 6 上：1rpx = 0.5px - Taro 自动将设计稿尺寸转换为 rpx

11.3 页面样式行数统计

文件	行数	核心样式
timeline/index.scss	453	时间轴、图表容器、指标卡片、数据输入弹窗
scan/index.scss	396	取景框、加载动画、结果卡片、药品编辑
profile/index.scss	358	用户卡片、可折叠分区、设置项、弹窗
home/index.scss	330	仪表盘、进度环、统计卡片、提醒列表
onboarding/index.scss	295	向导步骤、功能列表、头像选择器
medication/index.scss	206	药物卡片、时间表、库存进度条
box-scan.scss	150	取景框、加载、结果表单（独立）
Tutorial/index.scss	146	覆盖层、教程卡片、进度点
app.scss	83	全局变量、字体缩放、Reset
add.scss	78	添加表单、预设按钮、底部固定按钮
总计	**~2,495**	

---

12. 构建与部署

12.1 开发环境

```
# 安装依赖
npm install

# 启动开发模式（微信小程序）
npm run dev:weapp
# 或
npx taro build --type weapp --watch
```

```
# 输出目录：dist/
# 用微信开发者工具打开 dist/ 目录预览
```

12.2 生产构建

```
# 构建生产版本
npm run build:weapp
# 或
npx taro build --type weapp

# 构建产物在 dist/ 目录
```

12.3 部署流程

```
1. 本地开发并测试
      │
2. npm run build:weapp
      │
3. 微信开发者工具 → 上传
      │ 填写版本号和备注
      ▼
4. 部署云函数
      │ 右键 cloud/ocr → "上传并部署：云端安装依赖"
      │ 配置环境变量（TENCENT_SECRET_ID / KEY）
      ▼
5. 微信公众平台 → 版本管理 → 设为体验版
      │ 分发体验版二维码给测试人员
      ▼
6. 测试通过后 → 提交审核
      │ 填写功能描述、选择类目
      ▼
7. 审核通过 → 全量发布
```

12.4 Taro 构建配置

```
// config/index.ts
const config = {
  projectName: 'cc',
  date: '2025-2-28',
  designWidth: 750,          // 设计稿宽度 750px
  deviceRatio: {
    640: 2.34 / 2,
    750: 1,                   // 750 → 1:1 映射到 rpx
    375: 2,
    828: 1.81 / 2,
```

```
  },
  sourceRoot: 'src',
  outputRoot: 'dist',
  framework: 'react',
  compiler: 'webpack5',
  plugins: [],
  defineConstants: {},
  copy: { patterns: [], options: {} },
  mini: {
    postcss: {
      pxtransform: { enable: true, config: {} },
      url: { enable: true, config: { limit: 1024 } },
      cssModules: { enable: false },
    },
  },
}
```

---

13. 核心学习要点

13.1 Zustand 状态管理模式

要点 1：Immutable Update

```
// ✅ 正确：返回新对象
set((s) => ({ reminders: s.reminders.map(r => r.id === id ? { ...r, done:
true } : r) }))

// ❌ 错误：直接修改
set((s) => { s.reminders.find(r => r.id === id).done = true; return s })
```

要点 2：Derived State（派生状态） - `reminders` 是从 `medications`
派生的，不单独持久化 - 每次 medications 变化或 app 启动时重新计算

要点 3：persist 的 partialize - 不是所有 state 都需要持久化 - `family`
来自云端，不持久化 - `reminders` 是派生的，不持久化

13.2 React Hooks 设计模式

要点 1：自定义 Hook 封装订阅

```
// useT() 封装了对 store.lang 的订阅
// 使用它的组件会在 lang 变化时自动 re-render
export function useT() {
  const lang = useStore(s => s.lang)  // 选择性订阅
```

```
  return { t, tArray, lang }
}
```

要点 2：getState() vs useStore(selector) - `useStore(s => s.lang)` — Hook 方式，组件会 re-render - `useStore.getState().lang` — 直接读取，不触发 re-render - 在事件处理器、工具函数中用 `getState()`，在 render 路径上用 Hook

13.3 微信小程序特有知识

要点 1：Storage API - `Taro.getStorageSync(key)` — 同步读取，返回值或空字符串 - `Taro.setStorageSync(key, value)` — 同步写入 - 单个 key 最大 1MB，总量最大 10MB

要点 2：TabBar 动态更新 - `app.config.ts` 中的 TabBar 文字是编译时静态配置 - 运行时只能用 `Taro.setTabBarItem({ index, text })` 动态修改

要点 3：页面跳转 - `Taro.switchTab({ url })` — 跳转到 TabBar 页面 - `Taro.navigateTo({ url })` — 跳转到非 TabBar 页面（可返回） - `Taro.redirectTo({ url })` — 重定向（不可返回，替换当前页面） - `Taro.navigateBack()` — 返回上一页

要点 4：Canvas 2D - 小程序 Canvas 不是 Web 标准 Canvas，API 略有差异 - 需要通过 `Taro.createSelectorQuery()` 获取 Canvas 节点 - 必须处理 DPR（设备像素比）：`canvas.width = displayWidth * dpr`

13.4 OCR 正则解析技巧

要点 1：多关键词匹配

```
// 药品名可能在【通用名称】【药品名称】【商品名】任一标签下
text.match(/(?:通用名称|药品名称|商品名)[】：:\s]*([^\n【]+)/)
```

- `(?:...)` 非捕获组，用于多关键词匹配
- `[^\n【]+` 匹配到换行或下一个【之前的所有字符

要点 2：分段提取

```
// 用法用量段可能跨多行，用 [\s\S]*? 非贪婪匹配
text.match(/用法用量[】：:\s]*([\s\S]*?)(?=【|$)/)
```

- `[\s\S]*?` 匹配包括换行在内的任意字符（非贪婪）
- `(?=【|$)` 正向前瞻：到下一个【或文本结束为止

13.5 i18n 设计模式

要点 1：Flat Key 结构 vs 嵌套对象

```
// ✅ 本项目采用：Flat Key（扁平键）
{ 'home.todayProgress': '今日服药进度' }

// ❌ 另一种方案：嵌套对象
{ home: { todayProgress: '今日服药进度' } }
```

Flat Key 的优势： - 一行代码实现查找：`dict[key]` - 全文搜索更方便：搜索 `'home.todayProgress'` 即可找到所有使用处 - TypeScript 类型更简单

要点 2：不用 i18n 库的理由 - 整个 i18n 逻辑只有 ~20 行核心代码 - 引入 i18next / react-intl 会增加 ~30KB 包体积 - 不需要复数规则、日期格式化等高级功能 - 微信小程序对包体积敏感（主包 ≤ 2MB）

文档最后更新：2026-03-13